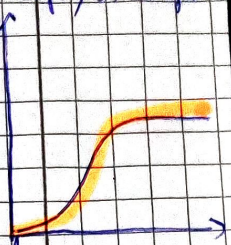
GMK – L1

Probabilistic Model Analysis

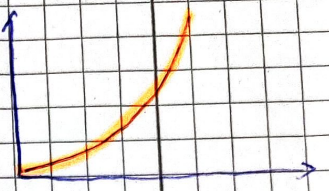
- Sigmoid Function: $f(x) = \frac{1}{1+e^{a(x-b)}}$

$a>0$: ↗, $a<0$: ↘, $b<0$: left, $b>0$: right



eg: Beschreibung Infektionszahlen von Pandemien, die anfänglich exponentiell wachsen ist dann unter Kontrolle gebracht.

- exponential function: $f(x) = a e^{bx} + c$



Probabilistic Models

Probabilistic Models
→ Model parameters are derived by statistics
→ Model descriptions are based on probabilities

Event:
- is a set of outcomes to which a probability is assigned
- we assume a sample space Ω
- Events: $A \subseteq \Omega$

Probability:
is a function P assigning a number to an event $A \subseteq \Omega$.
$P(A)$ = probability of event A and requires:
→ $0 \leq P(A) \leq 1$, $\forall A \subseteq \Omega$
→ $P(\Omega) = 1$
→ $P(A \cup B) = P(A) + P(B)$, $\forall A, B \subseteq \Omega$ with $A \cap B = \emptyset$

if A & B are not disjoint
$P(A \cup B) = P(A) + P(B) - P(A \cap B)$

GAK - 03

Probabilistic Models

Model:
a system that simulates and represents or approximates an object under consideration

Probabilistic Model:
a model that produces different outcomes with different probabilities.
→ simulate / represent / approximate a whole class of objects, assigning each a probability

eg: DNA sequences (objects) and a model describe a family of related sequences.

eg: Given sequence x of length n:

$$P(X) = \prod_{i=1}^{n} q_{+x[i]}$$

Events, often from different sample spaces whose occurrence depends on each other or which occur together.

Conditional Probability: Bedingte Wahrscheinlichkeit
Probability of some event X, given the occurrence of some other event Y.

$$P(X|Y) = \frac{P(X \cap Y)}{P(Y)}$$

eg: sample space for Dice $\{D_1, D_2\}$
P of rolling i with D_1: $P(i|D_1)$

Joint Probability: Kombinierte Wahrscheinlichkeit
Probability of two events occurring together. $P(X, Y)$
$P(X, Y) = P(X|Y)\,P(Y)$ eg: Picking D_j and rolling i $\Rightarrow P(i, D_j)$

$$P(X,Y) = P(X|Y)P(Y) = \frac{P(X \cap Y)}{P(Y)} P(Y) = P(X \cap Y) = P(Y \cap X) = P(Y|X)P(X) = P(Y,X)$$

GMK

Maximum Likelihood Estimation

Probabilistic Models have parameters : eg: f_a for $a \in \Sigma$ for DNA seq's

↳ subsumed in Θ

- $P(D|\Theta)$ the probability of observing any outcome D given model Θ (sequences of events that could be generated by the model with parameters Θ)
- what if we do not know Θ
  ↳ choose values of Θ s.t. $P(D|\Theta)$ is maximized
- Θ becomes a variable and we want to determine Θ s.t. the function $\mathcal{L}$:

  $\mathcal{L}(\Theta|D) = P(D|\Theta)$ is maximized

  ↳ likelihood function

  → Parameters derived from data → training data

  → $\mathcal{L}(\Theta|D)$ function of Θ returning $P(D|\Theta)$

  eg: total length of all sequences n. → $f_a = \frac{f_a}{n}$ → how often a event
- resulting probabilities derived from observed frequencies, maximizes the total probability of the training data when it is generated by the model

⇒ MLE determines the parameters Θ of a model from data, s.t. the probability that the model generates the data is maximized

GIK

MLE Example Coin

Coin: p : probability of tossing a head

toss ℓ times, k be the number of heads

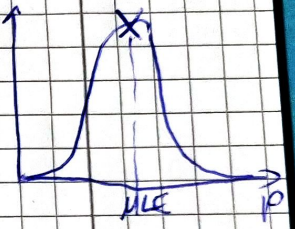
k positions out of ℓ which are marked as head

↳ $\binom{\ell}{k} = \frac{\ell!}{k!(\ell-k)!}$, possible sequences of ℓ tosses with k heads

↳ each has prob: $p^k(1-p)^{\ell-k}$

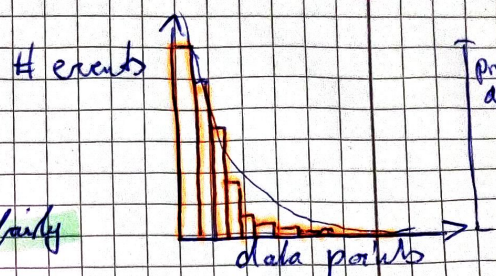
↳ $q(p, \ell, k) = P(\text{toss } k \text{ heads out of } \ell \mid p) = \binom{\ell}{k} p^k (1-p)^{\ell-k}$

eg: $\ell = 20$, $k = 7$ $\Rightarrow$ MLE: $p = \frac{7}{20}$



GIK

Maximum MLE - Example



Estimation for traffic Delay

Given: $t_1, .., t_n$ (times) of delays of a fairly reliable public transport system

→ no negative delays

→ shape of histogram looks like exponential distribution where # of consecutive events falls by a constant rate.

→ histograms ⟶ continuous function, probability density function (PDF)

→ PDF of exponential function: $f(\lambda, x) = \lambda e^{-\lambda x}$, $\lambda > 0$

↳ curve starts at $(0, \lambda)$

↳ $\int_{-\infty}^{\infty} f(\lambda, x)\, dx = 1$

↳ P(point falls into interval $[a,b]$) $= \int_a^b f(\lambda, x)\, dx$

→ MLE: estimate λ for $t_1, .., t_n$

$$\mathcal{L}(\underbrace{\lambda}_{\text{var}} \mid \underbrace{t_1, .., t_n}_{\text{data}}) = P(t_1, .., t_n \mid \lambda) = \prod_{i=1}^{n} \lambda e^{-\lambda t_i}$$

$$= \lambda^n e^{-\lambda \sum_{i=1}^{n} t_i}$$

→ $\bar{t} = \frac{1}{n} \sum_{i=1}^{n} t_i$, mean

↳ $\mathcal{L}(\lambda \mid t_1, .., t_n) = \lambda^n e^{-\lambda n \bar{t}}$ ↳ extreme light

↳ extreme overall

↳ calculate log-likelihood: max log-like. ⇒ max of like

↳ $\mathcal{L}(\lambda \mid t_1, .., t_n) = n \ln \lambda - \lambda n \bar{t}$ $\Big) \, d/d\lambda$

↳ $\mathcal{L}'(\lambda \mid t_1, .., t_n) = n \frac{1}{\lambda} - n\bar{t}$ $\Rightarrow \max \lambda = \frac{1}{\bar{t}}$

GIK

Prior - and Posterior Probability

eg: Casino 0,99 → fair Dice
0,01 → loaded Dice

→ picka die and roll [6,6,6] // loaded? Compare Models
→ Model D_{ld}, dice is loaded ($\frac{1}{2} = P(6)$)
→ Model D_{fr}, dice is fair
→ Determine which of the two models fits the observed data
better? → Calculate $P(D_{ld} | [6,6,6])$, the posterior probability that dice is loaded, given the observed data
→ prior probability is $P(D_{ld})$

Bayes Theorem

$$\underbrace{P(X|Y)}_{\text{posterior}} = \frac{\overbrace{P(Y|X)}^{\text{likelihood}} \overbrace{P(X)}^{\text{prior}}}{P(Y)}$$

$$P(Y) = P(Y|X)P(X) + P(Y|\neg X)P(\neg X)$$

eg:
$P([6,6,6] | D_{ld}) = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{8}$
$P([6,6,6] | D_{fr}) = \frac{1}{6} \cdot \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{6^3}$, $D_{fr} = \neg D_{ld}$
$P(D_{ld}) = \frac{1}{100}$
$P([6,6,6]) = P([6,6,6] | D_{ld}) P(D_{ld}) + P([6,6,6] | D_{fr}) P(D_{fr})$
$P(D_{ld} | [6,6,6]) = 0,21$ → more likely to pick fair
Which h s.t. loaded is more likely? $f(h) = P(D_{ld} | [6^h]) > 0,5$?

GMM

Bayes' Theorem and the EM Algorithm

Expectation Maximization: very general iterative method for estimating parameters of models from data

Expectation Maximization Algorithm

Mixture Models in 1-d → bad for many points

In: Observations $X_1, \ldots, X_n$ of experiment

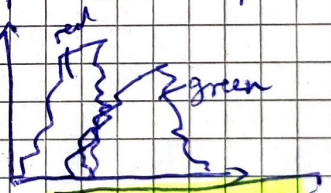
These observations can come from different sources!

Assume: 1) Observations generated from two normal distributions with unknown parameters (μ_r, σ_r) & (μ_g, σ_g)
(σ → standard deviation, μ → mean)

2) Coloured points, Source is known

eg: genome-wide localization of TF, clustering of gene expression points

→ Group similar values into bins and plot the # of values per bin as a bar ⇒ histogram



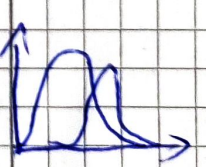
→ from points estimate parameters of the normals

→ MLE:

$$\mu_r = \frac{1}{|R|} \sum_{i \in R} x_i, \quad \sigma_r = \sqrt{\frac{\sum_{i \in R} (x_i - \mu_r)^2}{|R|}}$$

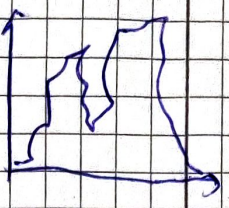
$$\mu_g = \frac{1}{|G|} \sum_{i \in G} x_i, \quad \sigma_g = \sqrt{\frac{\sum_{j \in G} (x_j - \mu_g)^2}{|G|}}$$

↳ applying those for given histograms → PDF



GIK

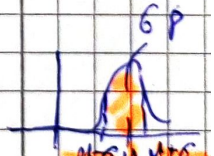
Mixture models 1-d, no coloring



→ two kinds of points by 2 normals
→ no known source
→ properties of the different points unknown
→ EM-Algorithm:
  to estimate the parameters of the normals and the proportion λ_r of red points

→ EM Alg (iterative Strategy):
  - expectation: from fixed model parameters Θ and known data estimate missing parameters
  - maximization: perform a MLE based on all parameters to find new parameters Θ'

→ convergence of parameters to local maxima ⇒ restart to improve result
→ suppose there are guessed parameters $\mu_r, \sigma_r, \mu_g, \sigma_g$ and λ_r
  ↳ fixed model parameters
→ EM estimates hidden parameter $P(r|x_i)$, $P(g|x_i) = 1 - P(r|x_i)$
→ first compute $P(x_i|r)$
  ↓ depends on both normals
  ↓ only depends on r-normal
→ $\phi(\mu, \sigma, x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$



→ P(point falls into interval $[a,b]$) $= \int_a^b \phi(\mu, \sigma, x)\, dx$
  ↳ whose bound axes are multiples of σ
  interval is fixed (6 8 - 9 5 - 9 9,7)
↳ $P(x_i|r) = P(\text{point falls into } [x_i - \varepsilon, x_i + \varepsilon]) = \int_{x_i-\varepsilon}^{x_i+\varepsilon} \phi(\mu_r, \sigma_r, x)\, dx$

GMK

Mixture models in 1-d, not ordering

→ $P(x_i | r)$ is called generative probability

→ $$P(r | x_i) = \frac{P(x_i | r) P(r)}{P(x_i | r) P(r) + P(x_i | g) P(g)}$$

$P(r) = \lambda_r$, $P(g) = 1 - \lambda_r$

→ $r_i = P(r | x_i)$, $\forall i \in [1, n]$

→ Update Parameters: $$\mu_r' = \frac{\sum_{i=1}^{n} r_i x_i}{\sum r_i}$$

$$\sigma_r' = \sqrt{\frac{\sum r_i (x_i - \mu_r')^2}{\sum r_i}}$$

$$\lambda_r' = \frac{1}{n} \sum r_i$$

→ $P(g | x_i) = 1 - P(r | x_i) = g_i$

→ Update Means: * point with large r_i has a larger effect on reshaping the normal

GMM

Mixture models in 1-d without colors

E.M:

1) start with random values for $\mu_r, \sigma_r, \mu_g, \sigma_g$ and λ_r
2) compute $P(r|x_i)$ according to Bayes' Theorem based on $P(x_i|r)$, $P(x_i|g)$, $P(r)$ and $\mu_r, \sigma_r, \mu_g, \sigma_g, \lambda_r$
3) $r_i = P(r|x_i)$,

$$\mu_r' = \frac{\sum r_i x_i}{\sum r_i}, \quad \sigma_r' = \sqrt{\frac{\sum r_i (x_i - \mu)^2}{\sum r_i}}, \quad \lambda_r' = \frac{1}{n} \sum r_i$$

4) $P(g|x_i) = 1 - P(r|x_i)$ compute analog μ_g', σ_g'

→ $\ln \mathcal{L}(\Theta | x_1, \ldots, x_n) = \ln \sum (P(x_i|r) + P(x_i|g))$ the log-likelihood of the pixels with Θ

→ if $\ln \mathcal{L}(\Theta | x_1, \ldots, x_n)$ already in differs from previous step, stop else next iteration! no improvement

GMK

Implementation of EM-Alg

-> Most Time consuming step:

$$P(x_i|r) = \int_{x_i-\varepsilon}^{x_i+\varepsilon} \phi(\mu_r, \sigma_r, x)\, dx$$

-> numerical method to compute the integral -> simple geometric units []

-> Mid point-method:

$[x_i-\varepsilon, x_i+\varepsilon]$ split into m subintervals of size $d = \frac{2\varepsilon}{m}$

$$P(x_i|r) \approx d \cdot \sum_{j=0}^{m-1} \phi(\mu_r, \sigma_r, m_j)$$

$m_j = x_i - \varepsilon + j \cdot d + \frac{d}{2}$ / midpoint of interval

$\hookrightarrow^{m=1}$ $P(x_i|r) \approx 2\varepsilon \cdot \phi(\mu_r, \sigma_r, x_i)$

$$\hookrightarrow P(r|x_i) = \frac{\phi(\mu_r, \sigma_r, x_i)\, P(r)}{\phi(\mu_r, \sigma_r, x_i)\, P(r) + \phi(\mu_g, \sigma_g, x_i) P(g)}$$

// no need of ε

// speeds up computation

EM

-> very general method for estimate parameters of models

can be used for:

-> for a style model with hidden variables

-> for more than two models

-> in higher dimensions

Modeling Context in Sequences

CpG-Islands

CpGs are rarer in the genome than would be expected from the independent probabilities of C and G.

CpG-islands: we see many more CpGs than elsewhere, useful marker.

↳ Questions:

1) Given a short segment of a genomic sequence, decide whether this segment comes from a CpG-island or not
2) Find all CpG-islands in a long genomic sequence

↳ Probabilistic Model

Markov Chains

Model CpG-Islands, large probabilities of CpG-islands and small probabilities for all other seq's.

→ Probability of one symbol depends on the probability of its predecessor.

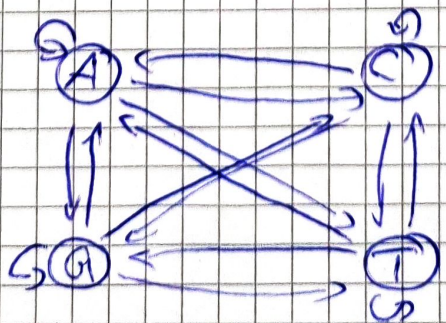
Markov Chain (time-homogeneous) (of order 1):

a pair (S, A):

→ $S = \{s_1, \dots, s_n\}$ is a finite set of states

→ $A = (a_{s,t})_{s,t \in S}$ is a transition matrix with values in range $[0,1]$, s.t. $\sum_{t \in S} a_{s,t} = 1, \quad \forall s \in S$

(S, A) generates a sequence X, s.t. $P(X_i = t \mid X_{i-1} = s) = a_{s,t}$ $\forall i$



label with $a_{s,t}$

$s \to t$

$$P(X) = P(X_1) \prod_{i=2}^{L} a_{X_{i-1}, X_i}$$

GIK -

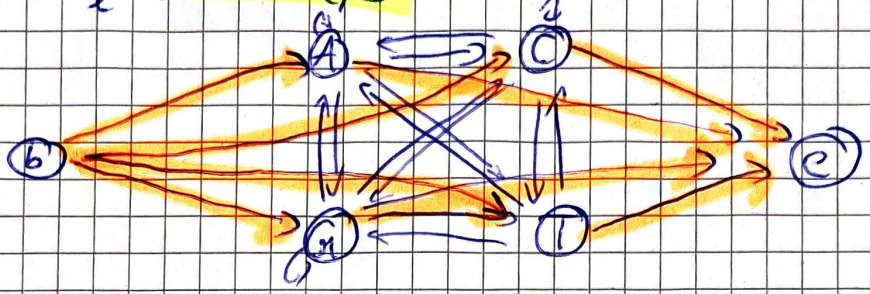
Modify the beg & end state

→ Add begin state b to the Markov Model, $X_0 = b$

→ $P(X_1 = S) = a_{b,s} = P(S)$, where $P(S)$ is the BG probability of state S.

→ End state e

→ $P(e \mid X_l = t) = a_{t,e}$



Determine the transition Matrix

→ set of DNA seq's, divided into two disjoint subsets

→ first set labeled as CpG, one labled CpG-islands

→ the other ¬CpG

Positive Model: transition matrix A^+ for DNA seq in CpG is:

$$a^+_{s,t} = \frac{c^+_{s,t}}{\sum_{t'} c^+_{s,t'}}$$

, $c^+_{s,t}$: # of positions in seq's where s is followed by t

negative model: analog for ¬CpG

CpG

Decide whether a ~~region~~ sequence comes from a CpG-island

Sequence $x = (x_1, \dots, x_l)$

$$P(x \mid \text{model}+) = \prod_{i=1}^{l+1} a^{+}_{x_{i-1} x_i}, \quad x_0 = b \text{ and } x_{l+1} = e$$

$$P(x \mid \text{model}-) = \prod_{i=1}^{l+1} a^{-}_{x_{i-1} x_i}, \quad x_0 = b \text{ and } x_{l+1} = e$$

log-odds scores:

$$S(x) = \log \frac{P(x \mid \text{model}+)}{P(x \mid \text{model}-)} = \sum_{i=1}^{l+1} \log \frac{a^{+}_{x_{i-1} x_i}}{a^{-}_{x_{i-1} x_i}}$$

$$\beta_{b,d} = \log \frac{a^{+}_{b,d}}{a^{-}_{b,d}}, \quad S(x) = \sum_{i=1}^{l+1} \beta_{x_{i-1} x_i}$$

↳ larger if larger importance

⇒ the higher log-odds score $S(x)$, the higher the probability, that x is a CpG-island

→ Normalize: $\frac{S(x)}{|x|} \gtrless 0$ → more likely CpG island

GIK

Hidden Markov Model

→ How to detect CpG-islands inside a long sequence

→ Windows would be used: uniform window size

⇒ Merge the two models + & −

↳ diffeal states A+ & A−

→ distinguish the sequence of states from the sequence of symbols

→ Probability of the state sequence:

similar as in Markov Chain, depends only on previous state

→ Probability of the symbol sequence: depends on the state sequence

↳ $e_k(b)$: Probability that symbol b is emitted in state k

→ sequence of symbols is known, sequence of states is hidden

Hidden Markov Model (HMM):

a quadruple $M = (\Sigma, Q, A, e)$:

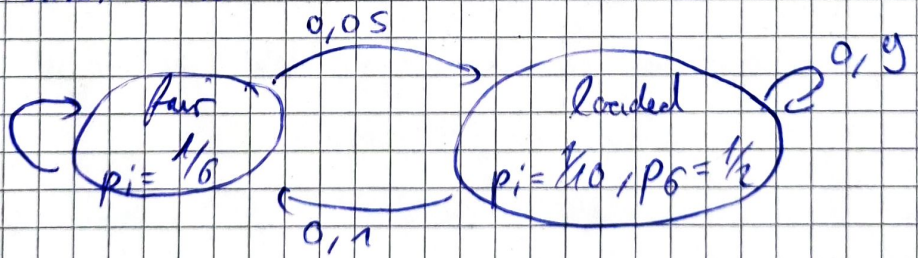
→ an alphabet Σ

→ a set of states Q

→ a matrix $A = (a_{s,t})_{s,t \in Q}$ of transition probabilities, $\forall s,t \in Q$

→ emission probabilities $e_k(b)$, $\forall k \in Q, \forall b \in \Sigma$

eg: dishonest Casino



Outcome: 6 5 4 3 6 6

Sequence of states hidden.

GitK

Hidden Markov Model

→ use HMM to generate data (start in state b (alias 0))

↳ while not reached state 0 (alias e), choose a state using transition probability, choose a symbol via emission probability

Path:

$\pi = (\pi_1, \dots, \pi_\ell)$ a sequence of states in the HMM M

Joint Probability:

given sequence of symbols $x[1 \dots \ell]$ and a path $\pi = (\pi_1, \dots, \pi_\ell)$

$$P(x, \pi) = a_{0,\pi_1} \prod_{i=1}^{\ell} e_{\pi_i}(x[i])\, a_{\pi_i, \pi_{i+1}}, \quad \pi_{\ell+1} = 0$$

eg: Casilex CpG-island

a path determines which parts of the sequence x are classified as CpG islands, namely the max subpaths consisting of + only

~~[redacted]~~

Problems

M a HMM and x a sequence

decoding problem: determine the most probable (illegible) sequence of states in M generating x.

Viterbi-Algorithm

evaluation problem: determine the probability that M generates x, $P(x \mid M)$.

Forward-Alg or Backward-Alg

learning problem: given some set of seq's of symbols, train the parameters of M.

Baum-Welch-Alg

GIK

Viterbi Alg dynamic programming

→ Decoding Problem:

determine the path π^* that maximizes the probability of generating the sequence x of symbols

$$P(x, \pi) \leq P(x, \pi^*)$$

→ Idea: calculate the maximum probabilities for $\forall$ prefixes of x and $\forall$ states

→ $V(q, i)$: probability of the most probable path ending in state q and generating $x[1..i]$

Alg: In: HMM $M = (\Sigma, Q, A, e)$ and sequence $x[1..l]$

Out: p most probable path π^*

Init:
$$V(q, 0) = \begin{cases} 1 & , \text{if } q = 0 \\ 0 & , \text{else} \end{cases}$$

Iter: $\forall\ i \in [1, l]$ and $q \in Q$

$$V(q, i) = e_q(x_i) \max\{V(k, i-1)\, a_{k,q} \mid k \in Q\}$$

$$Pred(q, i) = k \in Q, \text{ where } k \text{ satisfies max}\uparrow$$

Termination:

compute $P(x, \pi^*) = \max\{V(k, l)\, a_{k,0} \mid k \in Q\}$

$\pi_l^* = k \in Q$ where k max$\uparrow$

Traceback:

$\forall\ i = l-1, \dots, 1$ compute $\pi_i^* = Pred(\pi_{i+1}^*, i)$

→ Computes table of size $O(|Q| \cdot l)$

→ Each entry $O(|Q|)$

⇒ Running Time $O(|Q|^2 \cdot l)$

→ normally use log instead of multiplying → and sum up

GIK

HMM

-> $P(x|M)$, probability that x was generated by HMM

-> $P(x|M) = \sum_{\text{path } \pi} P(x, \pi | M)$

↳ we sum up the probabilities over all possible paths through M

Forward Algorithm

-> forward table $\mathcal{F}_x$: $\mathcal{F}_x(q,i) = P(x[1..i], \pi_i = q)$

↳ probability that the model generates the prefix $x[1..i]$ and reaches state $\pi_i = q$

Alg: In: HMM and sequence $x[1..l]$

Out: $P(x|M)$

Init: $\mathcal{F}_x(q,0) = \begin{cases} 1 & \text{, if } q = 0 \\ 0 & \text{, else} \end{cases}$

Iteration: $\forall\, i \in [1,l]$ and $q \in Q$

$$\mathcal{F}_x(q,i) = e_q(x[i]) \sum_{k \in Q} \mathcal{F}_x(k, i-1)\, a_{k,q}$$

Termination: $P(x|M) = \sum_{k \in Q} (\mathcal{F}_x(k,l)\, a_{k,0})$

GdK

Backward Algorithm

→ Backward-table B_x

$B_x(q,i) = P(x[i+1..l], \pi_i = q)$

↳ the probability to start in state $\pi_i = q$ and then generate the suffix $\pi[i+1..l]$ of x

Alg: In: HMM and sequence $x[1..l]$

Out: $P(x|M)$

Init: $B_x(q,l) = a_{q,0}$, $\forall q \in Q$

Ite: $\forall i = l-1, \dots, 1$ and $q \in Q$

$$B_x(q,i) = \sum_{k \in Q} (a_{q,k}\, e_k(x[i+1])\, B_x(k,i+1))$$

Term:

$$P(x|M) = \sum_{k \in Q} (a_{0,k}\, e_k(x[1])\, B_x(k,1))$$

Three Tables

I) $V(q,i)$ = Probability of the most probable path generating the symbol sequence $x^{[1..i]}$ and the HMM is in state q

II) $F_x(q,i)$ = Probability that the symbol sequence $x[1..i]$ is generated and the HMM is in state q

III) $B_x(q,i)$ = Probability that the HMM is in state q and then the symbol sequence $x[i+1..l]$ is generated

GIIK

$P(x,y) = P(x|y)P(y)$

Posterior Probabilities

→ F(w)-k B(w) - Alg are complementary.

prefix x and ending in q, start in q and suffix x

↳ Combination of both algorithms allows to restrict to paths which reach a specific state at position i when generating x

→ $P(\pi_i = q|x)$, probability that symbol $x[i]$ is generated in state $\pi_i = q$ given that sequence x is generated

$$P(\pi_i = q|x) = \frac{P(\pi_i = q, x)}{P(x)} = \frac{F_x(q,i)B_x(q,i)}{P(x)}$$

→ if $P(x,\pi) \approx P(x,\pi^*)$ → do not use Viterbi use alternative

→ $\hat{\pi}$ (sequence of states):

$\hat{\pi}_i = q$, if $P(\pi_i = q|x) \geq P(\pi_i = q'|x)$, $\forall q' \in Q, \forall i \in [1,L]$

↳ at any position choose the most probable state for that position

→ $P(\pi_i = q|x)$ is, if we are interested in a property of a position derived from the probabilities of a subset of states at that position

MSA if $q_h, q = 0$ ↳ coiled-coils

eg: CpG island detection

$$G(i|x) = \sum_{q \in Q^+} P(\pi_i = h|x), \quad Q^+ = \{t_+, G_+, C_+, A_+\}$$

↳ posterior probability that position i is in CpG-island

Gilk

Training the parameters

→ Problem of constructing an HMM

→ 1) determine topology : # states and connections

→ 2) determine parameters : $a_{k,q}$ & $e_k(b)$

→ In : Training Set

→ Goal : set the parameters in such a way that the probability that the given training seq's are generated by the HMM is maximized

Case 1 : symbol & state sequences known $x_1, \dots, x_n$ & $\pi_1, \dots \pi^n$

→ Choose parameters → Best explain Data

$$P(x_1, \dots, x_n, \pi^1, \dots, \pi^n \mid M = (\Sigma, Q, A, e)) \geq P(x_1, \dots, x_n, \pi^1, \dots, \pi^n \mid M = (\Sigma, Q, A', e')) \quad \forall A', e'$$

→ we set parameters : # $k \to q$ in seq !! ; # k produces b

$$\bar{a}_{k,q} = \frac{A_{k,q}}{\sum_{q' \in Q} A_{k,q'}} \quad , \quad \bar{e}_k(b) = \frac{E_k(b)}{\sum_{c \in \Sigma} E_k(c)}$$

Problem : overfitting

if $k \to q$ never seen $\Rightarrow \bar{a}_{k,q} = 0$

or if k never seen $\bar{e}_k(b)$ undefined

↳ Solution : introduces pseudocounts $r_{k,q}$ & $r_k(b)$

GIK

Baum-Welch-Alg

→ never à stable sequence unknown

→ more realistic

→ $P(x_1, .., x_n \mid M = (\Sigma, Q, A, e)) \geq P(x_1, .., x_n \mid M' = (\Sigma, Q, A', e'))$ (choose)

→ NP-Hard!

log-likelihood:

sequences are independent: $P(x_1, .., x_n) = P(x_1) \cdot ... \cdot P(x_n)$

log-likelihood of the model M w.r.t sequences $x_1 .. x_n$

$\sigma(x_1, .., x_n) = \log P(x_1, .., x_n \mid (A, e)) = \sum_{j=1}^{n} \log P(x_j \mid (A, e))$

↳ $P(x_j \mid (A, e))$ can be evaluated by F(j) or B(j)-Alg

→ good to choose (A, e) that maximize the log-likelihood.

GdK

Baum-Welch-Alg

Alg:

In: Alphabet Σ, state set Q of an HMM, sequences $x_1, \dots, x_n$ and pseudocounts $\tau_{k,q}$ and $\tau_k(b)$

Out: $M = (\Sigma, Q, A, e)$ // HMM

Init: set all entries in A and e to random probabilities or to identical probabilities.

1) $\forall j \in [1,n]$ compute $\overrightarrow{f_{x_j}}(q,i)$ and $\overleftarrow{b_{x_j}}(q,i)$ $\forall q \in Q$ and $i \in [0, |x_j|]$

2) $\forall j \in [1,n]$ add the contribution of x_j to A and E:

// # trans $k \to q$

$$\overline{A_{k,q}} = \sum_{j=1}^{n} \frac{1}{P(x_j)} \sum_{i=1}^{|x_j|} \overrightarrow{f_{x_j}}(k,i)\, a_{k,q}\, e_q(x_j[i+1])\, \overleftarrow{b_{x_j}}(q,i+1)$$

// # times b is emitted in k

$$\overline{E_k(b)} = \sum_{j=1}^{n} \frac{1}{P(x_j)} \sum_{i \in [1,|x_j|],\, x_j[i]=b} \overrightarrow{f_{x_j}}(k,i)\, \overleftarrow{b_{x_j}}(k,i)$$

3) Calculate new model parameters

$$\overline{a_{k,q}} = \frac{\overline{A_{k,q}}}{\sum_{q' \in Q} \overline{A_{k,q'}}} \quad \& \quad \overline{e_q(b)} = \frac{\overline{E_q(b)}}{\sum_{c \in \Sigma} \overline{E_q(c)}}$$

4) Set $A \leftarrow (\overline{a_{k,q}})_{k,q \in Q}$, $e \leftarrow (\overline{e_q(b)})_{k \in Q, b \in \Sigma}$

5) Compute log-likelihood $\sigma(x_1, \dots, x_n \mid (A,e))$

Terminate when log-likelihood did not improve or max # iterations

$\Rightarrow$ Converges to local maxima

↳ run several times to reach global maximum

Aufgabe 1

(10)

zwei Münzen p & p' für Kopf.

ℓ Würfe, k Köpfe

a) $p = \frac{1}{5}$, $p' = \frac{2}{5}$, $\binom{\ell}{k} = \frac{\ell!}{k!(\ell-k)!}$

$g(p, \ell, k) = P(\text{from } k \text{ heads out of } \ell \mid p) = \binom{\ell}{k}\left(\frac{1}{5}\right)^k \left(\frac{4}{5}\right)^{\ell-k}$

$g(p', \ell, k) = P(\text{from } k \text{ heads out of } \ell \mid p') = \binom{\ell}{k}\left(\frac{2}{5}\right)^k \left(\frac{3}{5}\right)^{\ell-k}$

$g(p, \ell, k) \leq g(p', \ell, k) \quad /\!/ \cdot \binom{\ell}{k}$

$\left(\frac{1}{5}\right)^k \left(\frac{4}{5}\right)^{\ell-k} \leq \left(\frac{2}{5}\right)^k \left(\frac{3}{5}\right)^{\ell-k} \quad /\!/ \log$

$k \log\left(\frac{1}{5}\right) + (\ell-k)\log\left(\frac{4}{5}\right) \leq k \log\left(\frac{2}{5}\right) + (\ell-k)\log\left(\frac{3}{5}\right)$

$\Leftrightarrow$

$$k \leq \ell \cdot \frac{\log\left(\frac{3}{5}\right) - \log\left(\frac{4}{5}\right)}{\log\left(\frac{1}{5}\right) - \log\left(\frac{4}{5}\right) - \log\left(\frac{2}{5}\right) + \log\left(\frac{3}{5}\right)}$$

then $\rightarrow p'$ else p

Which coin more likely used?

GMK

(10.3)

$x = e^{\log x}$, $\log(a \cdot b) = \log a + \log b$

$a \cdot b = e^{\log(a \cdot b)} = e^{(\log a + \log b)}$

$$P(x, \pi) = a_{0,\pi_1} \prod_{i=1}^{l} e_{\pi_i}(x[i]) \, a_{\pi_i, \pi_{i+1}}$$

$$e^{\log P(x,\pi)} = \cancel{\text{[illegible]}}$$

$$= e^{\log\left(a_{0,\pi_1} \prod_{i=1}^{l} e_{\pi_i}(x[i]) \, a_{\pi_i,\pi_{i+1}}\right)}$$

$$= e^{\log(a_{0,\pi_1}) + \sum_{i=1}^{l} \log(e_{\pi_i}(x[i]) \, a_{\pi_i,\pi_{i+1}})}$$

$$= e^{\log(a) + \sum_{i=1}^{l} \log(e) + \log(a)}$$

Viterbi

$\log(V(q,i)) = \log(e) + \max\{\log(V) + \log(a)\}$

return e^P

GIK – TA 2

(2.1)

$$P(ext|x) = \frac{\left(\prod_{i=1}^{n} f_{x[i]}^{ext}\right) P(ext)}{\left(\prod_{i=1}^{n} f_{x[i]}^{ext}\right) P(ext) + \left(\prod_{i=1}^{n} f_{x[i]}^{int}\right) P(int)}$$

$$= \frac{\left(\prod_{i=1}^{n} f_{x[i]}^{ext}\right) P(ext)}{\left(\prod_{i=1}^{n} f_{x[i]}^{ext}\right) P(ext)} \cdot \frac{1}{1 + \frac{\left(\prod_{i=1}^{n} f_{x[i]}^{int}\right) P(int)}{\left(\prod_{i=1}^{n} f_{x[i]}^{ext}\right) P(ext)}}$$

$$= \left(1 + \frac{\prod_{i=1}^{n} f_{x[i]}^{int} P(int)}{\prod_{i=1}^{n} f_{x[i]}^{ext} P(ext)}\right)^{-1}$$

$$= e^{\log\left(\left(1 + \frac{\left(\prod f_{x[i]}^{int}\right) P(int)}{\left(\prod f_{x[i]}^{ext}\right) P(ext)}\right)^{-1}\right)} = \exp\left(-\log\left(1 + \frac{\left(\prod f_{x[i]}^{int}\right) P(int)}{\left(\prod f_{x[i]}^{ext}\right) P(ext)}\right)\right)$$

$$= \exp\left(-\log\left(1 + e^{\log\left(\frac{\left(\prod f_x^{int}\right) P(int)}{\left(\prod f_x^{ext}\right) P(ext)}\right)}\right)\right)$$

$$= \exp\left(-\log\left(1 + \exp\left(\log\left(\left(\prod f_x^{int}\right) P(int)\right) - \log\left(\left(\prod f_x^{ext}\right) P(ext)\right)\right)\right)\right)$$

$\log\left(\left(\prod f_x^{int}\right) P(int)\right) = \log(P(int)) + \sum_{i=1}^{n} \log f_x^{int}$

$\log\left(\left(\prod f_x^{ext}\right) P(ext)\right) = \log(P(ext)) + \sum_{i=1}^{n} \log f_x^{ext}$

$$= \exp\left(-\log\left(1 + \exp\left(\hat{q} - \hat{p}\right)\right)\right)$$